

# PROJECT DETAILS JAN 2025

## DEADLINES:

1. Milestone 1 : Deadline by the end of Week 2 : Jan 28, 2025
2. Milestone 2: Deadline by the end of Week 4 : Feb 9, 2025
3. Milestone 3: Deadline by the end of Week 5 : Feb 20, 2025
4. Milestone 4: Deadline by the end of Week 8 : March 5, 2025
5. Milestone 5: Deadline by the end of Week 9 : March 20, 2025
6. Milestone 6: Deadline by the end of Week 10 : Apr 1, 2025

## Project Statement:

### AI Agent for Academic Guidance

The primary way in which learners engage with content in the IITM BS degree program is through the SEEK portal. The portal contains learning videos, assignments, resources, practice and graded assignments, and programming quizzes. The portal can be described as a learning environment and provides opportunities for self-learning, by enabling learners to learn at their own pace and schedule.

With the rapid advancement of generative AI (GenAI) technologies, there are promising possibilities of effective integration of GenAI into these learning environments. The goal of this project is to develop an AI agent that enhances the learning experience for students enrolled in the IITM BS program. The agent will serve as a virtual guide, helping students navigate the course material, improve their study habits, and adhere to academic integrity principles. Rather than providing direct answers or solutions, the agent will nudge students toward adopting better learning strategies and suggest resources to support their understanding. The agent will be designed to support various types of courses, including programming courses where solutions to practice assignments are accessible as part of the input data. The primary client for this project is the student

community, and the agent should foster a collaborative learning environment for them.

The agent should not provide direct answers to questions. Instead, it should suggest relevant learning strategies, direct students to reference materials, and guide them to appropriate course content. The agent should cater to the courses that students are enrolled in. Course-specific guidance should be dynamic, reflecting the practice assignments and resources available for each course. The agent should use contextually relevant resources to point students to existing documentation, tutorials, or discussions that address their queries. It should be capable of directing students to credible academic references and official materials.

The above examples are just to give you an idea about the features which are possible. You are free to use/not use the above examples and come up with other innovative ideas.

## Technical Constraints

### 1. Standard Conversational Features:

- The agent should include core features of modern conversational AI, such as:
  - Conversation threading to maintain context across interactions.
  - Artefacts storage for key interactions and recommendations.
  - Adherence to rate limits and a defined context window.

### 2. Retrieval-Augmented Generation (RAG) Architecture

- To handle common queries, the agent should implement a RAG system that leverages a pre-indexed knowledge base.
- This approach should minimize calls to the language model (LLM) for frequently asked questions.

### 3. Academic Integrity:

- The agent must enforce principles of academic ethics.
- It should highlight general strategies and avoid divulging solution specifics for assessments.

## PROJECT EVALUATION

### Project - 30 marks

- Milestone 1 - 3 : 10 marks
- Milestone 4 - 6 : 10 marks
- Presentation : 10 marks

Evaluation in each of the milestones:

- After Milestone-3, Peer Evaluation-1 (Milestones 1-3) and
- After Milestone-6, Peer Evaluation-2 (Milestones 4-6)
- Instructor evaluation (80%)
- Peer evaluation (20%)

### MILESTONE:1

- **Focus:** Identify User Requirements
- **Identify users** of the application - primary, secondary and tertiary users.
- Write **user stories** for the requirements, based on the SMART guidelines discussed in the lectures (You are not required to write user stories for existing functionalities; instead, you have to write them for the features and integrations you want to implement in the existing system)
- The user stories should be in the following format:
  - ◆ As a [type of user],
  - ◆ I want [an action],
  - ◆ So that [a benefit/value]

### PEER EVALUATION Jan 2025

#### PEER EVALUATION QUESTIONS:

- 1. Identify users (out of 5): All users identified - 5, Partially identified-3, Absent - 0
- 2. Write user stories (out of 15): Fully identified - 15, Partially identified (as per the SMART guidelines) - 10, Partially identified (but mostly not as per the SMART guidelines) - 5, Absent - 0

[Please check whether the user stories follow the SMART guidelines]

*The following image is for illustration purposes.*

## **Identifying the various types of Users**

**Primary Users:** Students, Support Staff, and Admins

**Secondary Users:** Managers

**Tertiary Users:**

- Software developers: A new feature request for the main IIT portal is highly upvoted. Thus Administrators/managers ask the software developers to implement that feature. Whilst they don't use the Support Desk, they are impacted by the use/decision of Primary and Secondary Users.
- The platform that hosts the website, the Internet Service Provider, etc.

## **User Stories**

1. - **As a student**
  - I want to be able to create new support/query tickets
  - So that I can get help with my issues from the support team
2. - **As a student**
  - I want an edit option to edit my ticket post submission and before resolution
  - so that I can convey myself better if there is a need.
3. - **As a student**
  - I want the ability to delete a previously submitted ticket by me
  - so that I do not hold the queue up if my query has been resolved by me already.
4. - **As a student**
  - Before submitting my query ticket, I want the system to show if similar query tickets based on the title or content have been raised previously or are in the FAQ section
  - So that I can "+1" the relevant ticket or directly resolve it if already answered.

*The following image is for illustration purposes.*

## **MILESTONE:2**

- **Focus:** User Interfaces
- Create a **storyboard** for the application - it can be a ppt or even a video.

Embed the ppt/video in the PDF submission of this week.

- Take each user story and create **low-fidelity wireframes**
- Apply usability design guidelines and heuristics discussed in lectures to come up with the wireframes.

## PEER EVALUATION Jan 2025

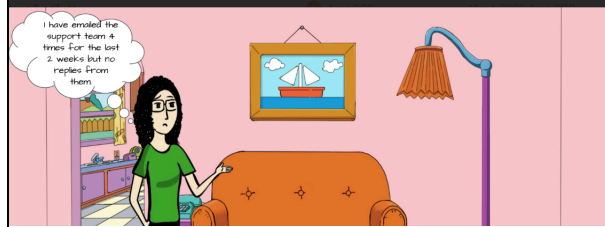
**3. Create a storyboard (out of 10):** All criteria satisfied with drawing - 10, Some criteria satisfied - 5, Absent - 0

**4. Create low-fidelity wireframes for the identified user stories (out of 15):** Fully available - 15, Mostly available - 10, A few available - 5, Absent - 0



# Storyboard 2

From the perspective of a student



*The following image is for illustration purposes.*

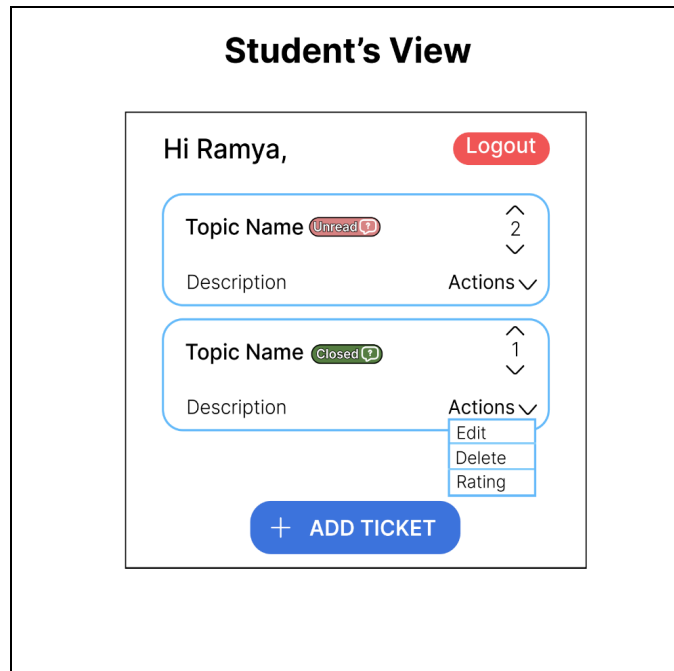
## LOGIN PAGE

### LOGIN

Email

Password

Submit



*The following image is for illustration purposes.*

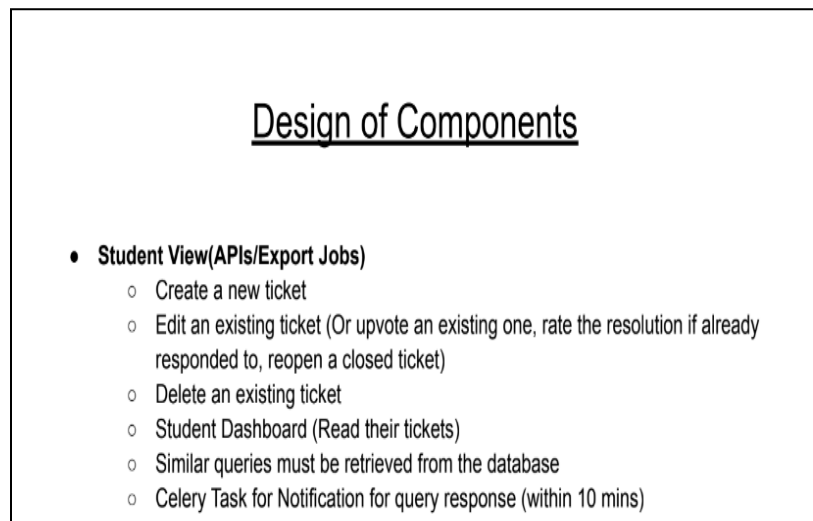
## MILESTONE:3

- **Focus:** Scheduling and Design
- **Project Schedule** - come up with a schedule of your overall project based on the user stories created in the previous milestones
- Create a schedule for your sprints and iterations, timings of your scrum meetings etc. Trello board, Gantt chart - specifying your tasks and contributions.
- **Project Scheduling Tools** - which tools are you using? E.g. Pivotal Tracker, Jira
- **Design of Components** - Describe different components of your system based on the user stories created in the previous milestones
- **Software Design** - Basic **class diagrams** of your proposed system
- **Details/Minutes of a few scrum meetings**
- Most of the pages in the **UI** (It may be modified (up to a certain limit) at a later time if necessary) - E.g.: HTML/CSS/Javascript (if you are using a framework like Vue.js or React.js, pages should be connected and redirection also implemented, have to submit the frontend folder without integration with backend API's. Additionally **provide a Readme file** to run the frontend code).

→ **Milestone 3 PDF Report**

**PEER EVALUATION Jan 2025**

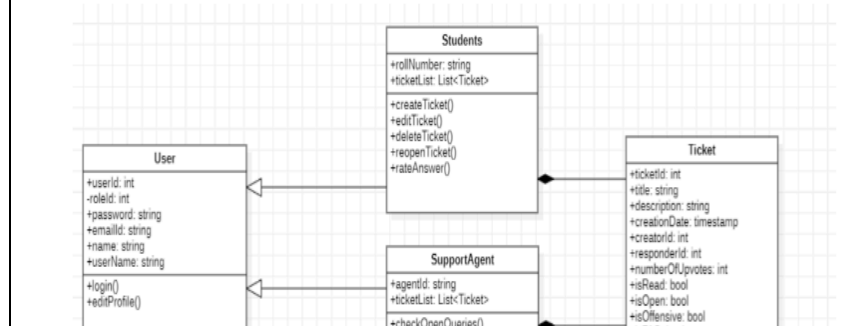
5. **Project Schedule (out of 7):** Full Gantt diagram and description present - 7, Partially present - 4, Absent - 0
6. **Use of project scheduling tools (out of 5):** Present - 5, Partially present - 3, Absent - 0
7. **Describe different components of your system (out of 8):** Components are fully identified with description - 8, Partially present - 4, Absent - 0
8. **Basic class diagrams of your proposed system (out of 10):** All classes with appropriate relationship - 10, Most classes with appropriate relationship - 7, A few classes with appropriate relationship - 5, A few classes with relationship which are not appropriate - 3, Absent - 0
9. **Details/Minutes of a few scrum meetings (out of 5):** Fully present - 5, Partially present - 3, Absent - 0
10. **UI pages (add screenshots) (out of 15):** All important pages - 15, Most of the pages - 10, A few pages - 5, Absent - 0
11. **Overall milestone 1-3 (out of 5):** Very impressive - 5, Impressive - 3, Poor - 0



*The following image is for illustration purposes.*



## Class Diagram



*The following image is for illustration purposes.*

## Sprint Schedule

- ❖ **Sprint 1:** Identify the types of users
  - Date: 06/02/2023 - 10/02/2023
- ❖ **Sprint 2:** SMART User Stories
  - Date: 11/02/2023 - 16/02/2023
- ❖ **Sprint 3:** Vetting & Submission
  - Date: 17/02/2023 - 19/02/2023
- ❖ **Sprint 4:** Storyboarding, Wireframe, Applying Usability Principles to Wireframe, Vetting Submission, Final Submission
  - Date: 20/02/2023 - 26/02/2023
- ❖ **Sprint 5:** Jira Roadmap Setup, Design of Components, UML Diagrams, Vetting Submission, Final Submission
  - Date: 27/02/2023 - 05/03/2023
- ❖ **Sprint 6:** Database Schema, Database Models, User Class, Students Class (Code Review, Database Review, Code Review)

*The following image is for illustration purposes.*

## SCRUM Meetings Schedule and Minutes

**SCRUM Meetings: Every Monday, Wednesday, and Friday 19:00-20:30**

**Minutes from Sprint 1 SCRUM Meetings:**

Identified and discussed the different types of users (primary, secondary, and tertiary). Discussed a few User Stories associated with the aforementioned users and agreed to come up with at least 10 user stories each.

**Minutes from Sprint 2 SCRUM Meetings:**

Discussed our User Stories and applied SMART Guidelines to refine the User Stories.

**Minutes from Sprint 4 SCRUM Meetings:**

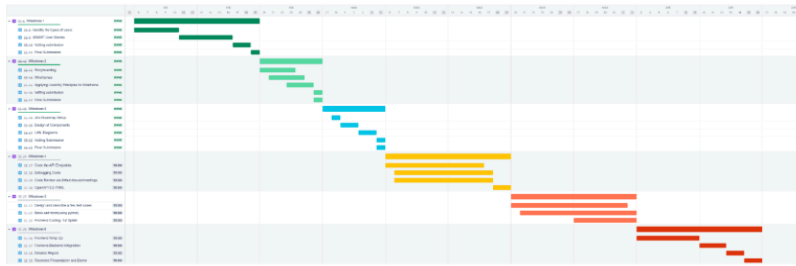
Discussed the software to design the storyboards and wireframes. Also worked on the initial draft of our wireframe. Chirag was tasked to mainly refine the wireframe; Varun and Arya were tasked to come up with a few storylines for our storyboards to discuss in our next SCRUM meeting. Discussed our storylines for our storyboards and agreed on a few to take further. Applied design heuristics and found our present software inadequate to express the same. Thus decided to switch to a different software to design our wireframe.

**Minutes from Sprint 5 SCRUM Meetings:**

Came up with a schedule for our project and an appropriate project scheduling tool, which was decided to be **Jira**. We then designed the components on pen and paper and finally used StarUML to produce the digital version of the same.

*The following image is for illustration purposes.*

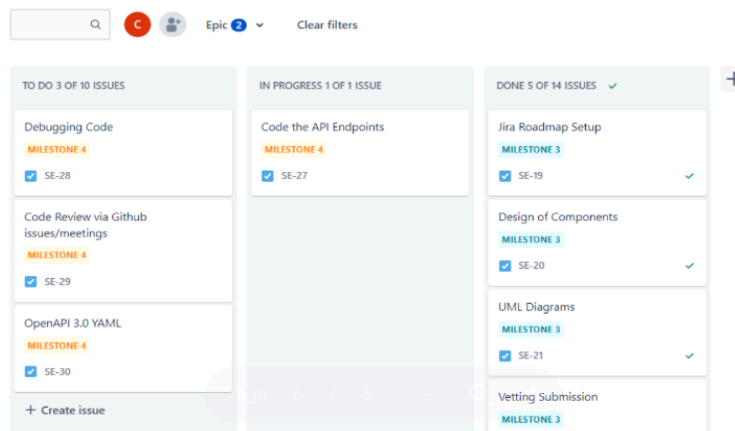
## GANTT Chart



## (Partial) Kanban Board

Projects / Software Engineering

### SE board



*The following image is for illustration purposes.*

## MILESTONE:4

- Focus: API Endpoints
- For each user story, create new API endpoints or use appropriate API endpoints from libraries
  - ◆ List of APIs integrated
  - ◆ List of APIs created
- Description of API endpoints. (As per the problem statement)
- **Submission of YAML (for the APIs created by dev-team) (When submitting YAML file for API, make sure it is Swagger compatible)**
- **Code for the APIs (implementation)**

## PEER EVALUATION QUESTIONS:

### 1. API Creation and integration (out of 15):

- Provide a detailed description of the APIs in the YAML file, ensuring all API's are clearly listed.
- Explain how the developed API's are linked with the user stories, specifying how each API helps to implement the different user stories provided in Milestone 1. If APIs (such as GenAI API's) have been integrated to create new ones, describe their usage in the next section.

YAML contains all of the required API's and they are mapped to all the user stories. Additionally , any integrated APIs from GenAI (created by dev-team) are clearly listed - 15, YAML has some required APIs and some user stories are implemented, and integrated and GenAI Api's listed- 8, YAML has some required API, but it is not formatted properly and the user stories not implemented, GenAI API's not listed - 3, Absent - 0

### 2. Code for the APIs - Implementation (out of 20):

- Provide the complete code for the APIs that have been implemented.
- Ensure that the code is well commented and with proper error handling, validation and responses. The implementation should match the YAML and User stories.

Code for all APIs are present, well documented adheres the best practices (error handling, validation and responses), fully implements the user stories - 20, Code for all the APIs is according to YAML and user stories but best practices (error handling, validation and responses) not followed- 15, Code is incomplete, poorly formatted, lacks proper documentation, but according to User Stories - 10 Code is incomplete, poorly formatted, lacks proper documentation and failed to implement User Stories - 5, Absent - 0

## MILESTONE:5

- **Focus:** Test cases, test suite of the project
- For each API endpoint, design extensive test cases. Test cases should be in the following format:
  - [ API being tested,
  - Inputs,
  - Expected output,
  - Actual Output,
  - Result- Success/Fail ]

### PEER EVALUATION QUESTIONS:

#### 3. Design and describe extensive test cases (out of 20):

- Test cases for APIs created.
- Test cases for other functionalities.

Most of the important test cases with proper format - 20, Some of the important test cases with proper format - 15, Some of the important test cases (not formatted properly)- 10, Test cases are not correct - 5, Absent - 0

#### 4. Some basic unit tests using pytest (out of 5):

pytest code/output present - 5, pytest code/output present, but error are not highlighted - 3, Absent - 0

**Sample screenshots based on the previous term's problem statement:**

A new item suggested by a support agent is approved/rejected by the admin for FAQ

**Page being tested:** http://127.0.0.1:5000/api/faq

**Inputs:**

- Request Method: POST
- JSON: {"category": "operational", "is\_approved": false, "ticket\_id": 2}
- Header: secret\_authtoken: abcxyz

**Expected Output:**

- HTTP Status Code: 200
- JSON: {"message": "FAQ item added successfully"}

**Actual Output:**

- HTTP Status Code: 200
- JSON: {"message": "FAQ item added successfully"}

**Result:** Success

```
def test_faq_authorized_role_post_valid_data():
    input_dict = { "category": "operational", "is_approved": False, "ticket_id": 2}
    data = json.dumps(input_dict)
    header={"secret_authtoken":token_login_admin(), "Content-Type":"application/json"}
    request=requests.post(url_faq,data=data, headers=header)
    assert request.status_code==200
    assert request.json()['message']=="FAQ item added successfully"
    faq = FAQ.query.filter_by(ticket_id=2).first()
    assert input_dict["category"] == faq.category
    assert input_dict["is_approved"] == faq.is_approved
```

An existing ticket in the FAQ is updated with a new category

**Page being tested:** http://127.0.0.1:5000/api/faq

**Inputs:**

- Request Method: PATCH
- Json body: {"category": "random", "is\_approved": false, "ticket\_id": 2}
- Header: secret\_authtoken: abcxyz

**Expected Output:**

- HTTP Status Code: 200
- JSON: {"message": "FAQ item updated successfully"}

**Actual Output:**

- HTTP Status Code: 200
- JSON: {"message": "FAQ item updated successfully"}

**Result:** Success

```
def test_faq_authorized_role_patch_valid_data():
    input_dict = { "category": "random", "is_approved": False, "ticket_id": 1}
    data = json.dumps(input_dict)
    header={"secret_authtoken":token_login_admin(), "Content-Type":"application/json"}
    request=requests.patch(url_faq,data=data, headers=header)
    assert request.status_code==200
    assert request.json()['message']=="FAQ item updated successfully"
    faq = FAQ.query.filter_by(ticket_id=1).first()
    assert input_dict["category"] == faq.category
    assert input_dict["is_approved"] == faq.is_approved
```

## MILESTONE:6

- **Focus:** Final Submission pdf
- Week-12 is completely focused on the project, and no new course content is released this week
- Complete **implementation** along with a working prototype.

- Final project report (consistent with intermediate milestone documents).
- Detailed report on work done from Milestone 1 through Milestone 5.
- Implementation details of your project
  - ◆ 1. Technologies and tools used
  - ◆ 2. And instructions to run your application.
- A section describing code review, issue reporting and tracking using screenshots.
- Recorded presentation and presentation slides of the working model of your system.

## **PEER EVALUATION QUESTIONS:**

### **5. Presentation and recorded demo of your application(out of 5):**

Submitted - 5,  
Not submitted - 0

### **6. Tool & Technology (Out of 15):**

All tools and technologies are presented - 15,  
Most of the tools and technologies are presented - 10,  
Some of the tools and technologies are presented- 5,  
Absent - 0

### **7. Issue tracker (Out of 10):**

Issue trackers are present with regular updates- 10, Issue trackers are present with irregular updates - 5, Absent -0

### **8. Instructions to run your application (Out of 5):**

Available in full details - 5, Some information available - 3, Absent - 0

### **9. Overall milestone 4-6 (out of 5):** Very impressive - 5, Impressive - 3, Poor - 0

## **Implementation Details of the Project**

- **Technologies and tools used**

- **Technologies for the backend**

- Flask
    - Flask Restful (For creating API endpoints)

- .....

- **Technologies for the frontend**

- Vue 3 CLI
    - JavaScript
    - Vue Router

- ....

- **Technologies for GenAI integration**

- OpenAI API
    - Gemini API
    - HuggingFace API
    - LangChain

- ....

- **General technologies used**

- GitHub (for versioning, code management, tracking, reviewing, issues, etc.)
    - Algolia Search is used in the backend and frontend to create a smooth search experience
    - Jira (for project management)

- ....

- **Hosting:** info regarding hosting

- **Instructions to run the application**

- On Ubuntu/MAC OS:

- Git clone the repository.
    - Change the directory to the “backend” directory inside the “Milestone-6-Final-Submission” directory using the command :  
cd ./Milestone-6-Final-Submission/Code/backend
    - Create a Python virtual environment using the command:  
python3 -m venv “<<Name of the virtual environment>>”
    - Activate the virtual environment using the command:  
source <<Name of the virtual environment>>/bin/activate
    - Install the requirements using the command :  
pip3 install -r requirements.txt.....**and so on**



- On Windows:
- Git clone the repository.
- Change the directory to the “backend” directory inside the “Milestone-6-Final-Submission” directory using the command :  
`cd .\Milestone-6-Final-Submission\Code\backend...`**and so on**

## Code Review, Issue Reporting and Tracking

This was completely done on GitHub.

### ISSUES:

The screenshot shows the GitHub Issues interface for the repository 'bsc-itm / soft-engg-project-jan-2023-group-1-1'. The page displays a list of 12 issues, with 2 open and 10 closed. The issues are listed in a table with columns for Author, Label, Projects, Milestones, Assignee, and Sort. The issues are as follows:

| Issue Title                                                                    | Author              | Label       | Projects | Milestones | Assignee | Sort |
|--------------------------------------------------------------------------------|---------------------|-------------|----------|------------|----------|------|
| two different references of datetime imported in api.py                        | #50 by apurva-kumar |             |          |            |          | 1    |
| Regarding the Py/WT structure                                                  | #59 by apurva-kumar |             |          |            |          | 1    |
| Regarding Body in GET requests                                                 | #58 by apurva-kumar |             |          |            |          | 1    |
| Regarding autoincrement = True for ticket_id in FAQ model class                | #57 by apurva-kumar | help wanted |          |            |          | 2    |
| getResolutionTimes Endpoint needs a few minor corrections                      | #53 by 2710302743   |             |          |            |          | 2    |
| Are all tables necessary                                                       | #52 by Chirag-Gupta | question    |          |            |          | 1    |
| PATCH request for TicketsAPI Endpoint should allow the students to give rating | #51 by apurva-kumar |             |          |            |          | 2    |
| Relationship of student/support agent class with ticket class                  | #50 by 2710302743   | question    |          |            |          | 1    |
| Missing components in the first UML diagram                                    | #48 by apurva-kumar | help wanted |          |            |          | 2    |
| Repository needs restructuring to comply with project instructions             | #47 by 2710302743   | help wanted |          |            |          | 2    |
| Issues with First Wireframe on Escalidraw                                      | #42 by Chirag-Gupta | wireframe   |          |            |          | 1    |

At the bottom of the page, there is a note: 'NoTgl included in the last three days updated - 2023-04-12'.

## Pull Requests:

The screenshot shows the GitHub Pull Requests interface for the 'ispe' repository. The top navigation bar includes links for Pull requests (2), Actions, Projects, Wiki, Security (4), and Insights. Below the navigation bar, there's a search bar with the text 'ispe is closed', a 'Labels' button (11), a 'Milestones' button (0), and a 'New pull request' button. A 'Clear current search query, filters, and sorts' link is also present. The main content area displays a list of pull requests with columns for status (Open/Closed), title, author, label, projects, milestones, reviews, assignee, and sort. The list includes several pull requests, some of which are merged and approved.

| Status | Title                     | Author                | Label | Projects | Milestones | Reviews               | Assignee | Sort |
|--------|---------------------------|-----------------------|-------|----------|------------|-----------------------|----------|------|
| 2 Open | 40 Closed                 |                       |       |          |            |                       |          |      |
| Merged | Update openapi.yaml       | #54 by 21f1000743     |       |          |            | Approved              |          | 3    |
| Merged | Update openapi.yaml       | #53 by 21f1000743     |       |          |            | Approved              |          | 3    |
| Merged | amin frontend corrections | #52 by Chirag-Goel-17 |       |          |            | was merged 3 days ago |          |      |
| Closed | Chirag frontend           | #51 by Chirag-Goel-17 |       |          |            | was closed 3 days ago |          |      |
| Merged | Flagged Post tracking     | #50 by aryab-sudo     |       |          |            | was merged 4 days ago |          |      |
| Merged | Arya frontend 2           | #49 by aryab-sudo     |       |          |            | was merged 4 days ago |          | 2    |
| Merged | buttons display           | #48 by Chirag-Goel-17 |       |          |            | was merged last week  |          |      |
| Merged | support agent view        | #46 by Chirag-Goel-17 |       |          |            | was merged last week  |          | 3    |

## Code Reviews

The screenshot shows the GitHub Code Review interface for the 'Update openapi.yaml' pull request (#54). The top bar includes 'Edit' and '<> Code' buttons. Below the bar, there's a 'Conversation' tab (5), 'Commits' (3), 'Checks' (0), and 'Files changed' (1) tabs. A status bar shows '+158 -1' changes. The main content area displays a list of comments and reviews. A comment from '21f1000743' states 'Added missing documentation'. A review from 'Chirag-Goel-17' is shown, with a 'View reviewed changes' link. The review details show a diff for 'Intermediate work/code/backend/openapi.yaml' with 1328 additions and 1 deletion.

**Update openapi.yaml #54** Edit <> Code

Merged aryab-sudo merged 3 commits into main from varun\_yaml\_doc yesterday

Conversation (5) Commits (3) Checks (0) Files changed (1) +158 -1

21f1000743 commented 2 days ago

Added missing documentation

Update openapi.yaml 214/158

21f1000743 requested review from aryab-sudo and Chirag-Goel-17 2 days ago

Chirag-Goel-17 reviewed 2 days ago View reviewed changes

Intermediate work/code/backend/openapi.yaml Outdated

1328 + application/json:

## **Project Presentation**

- We will schedule a Software Engineering Project Showcase, where each group will present their project to instructors, other groups, and even other students in the BS program
- The presentation will be graded.
- Components evaluated in final Project Evaluation:
  1. The overall design of the project.
  2. Teamwork that includes code review, issue reporting and tracking etc.
  3. Efficient coding practices.
  4. Consistent design through different milestones.